

EJERCICIOS EXTRA

Ejercicios de Repaso para la Final

UNIDAD 1 – Programación de Comunicaciones en Red

Hay que saber o repasar:

- Qué es un puerto (diferenciarlo del concepto de puerto) y para qué sirve en la comunicación vía sockets
- Entender que la comunicación empleando sockets se puede realizar entre programas alojados en la misma máquina o máquinas diferentes
- Crear un programa que simule un servidor (por tanto, no finaliza nunca) que recibe peticiones por el puerto que se pida y da respuesta al cliente en función de la petición realizada
- Crear un programa que simule un cliente que:
 - Realiza la petición al servidor.
 - Espera la respuesta.
 - La procesa y finaliza.
- La información que se intercambian los programas deben ser objetos Serializables, repasar los apuntes de Moodle al respecto. Los objetos de la clase String lo son ya, no hace falta especificarlo.
- Ver en Moodle los recursos Cliente y Servidor.
- Manejo de HashMap, arrays y ArrayLists.
- Entrada-salida de teclado con System.out etc y la clase Scanner.
- El orden de las instrucciones importa en este caso...
 - Debe de cerrarse primero la salida y finalmente la entrada:

```
salida = new ObjectOutputStream(cliente.getOutputStream());  
entrada = new ObjectInputStream(cliente.getInputStream());
```

EJERCICIO 1

Se desea desarrollar un servidor Java que permita a los clientes interactuar con un sistema de control, sin usar hilos. El servidor debe cumplir las siguientes condiciones:

- 1. Puertos distintos para entrada y salida:**
 - a. El servidor escucha en un primer puerto para recibir peticiones de los clientes.
 - b. Todas las respuestas o actualizaciones derivadas de esas peticiones se envían a través de un segundo puerto.
- 2. Recepción y procesamiento secuencial:**
 - a. El servidor atiende a un cliente a la vez, recibiendo su petición, procesándola y enviando la respuesta antes de pasar al siguiente cliente.
 - b. Cada petición es procesada de manera secuencial, en el orden en que se reciben.
- 3. Envío de respuestas:**
 - a. Cada petición recibida genera una respuesta que se envía por el segundo puerto.
 - b. Por ejemplo:

- i. Si un cliente pide el estado actual de un sensor, el servidor envía el valor por el puerto de salida.
- ii. Si un cliente solicita iniciar un proceso, el servidor confirma por el puerto de salida y actualiza el estado interno.

Ejemplo de interacción esperada:

Cliente envía por puerto 5000: "CONSULTAR_TEMPERATURA"

Servidor procesa → envía por puerto 5001: "TEMPERATURA=23.5°C"

Cliente envía por puerto 5000: "INICIAR_MOTOR"

Servidor procesa → envía por puerto 5001: "MOTOR=ACTIVO"

UNIDAD 2 – Programación Multihilo

Hay que saber crear hilos de las 2 formas vistas:

- Clase hija de Thread
- Implementar Runnable
- No se puede llamar al método run para lanzar la ejecución del hilo porque entonces se ejecuta de forma secuencial, en lugar de simultáneamente, como se pretende.

EJERCICIO 1

Haz un programa que muestre 3 número aleatorios que cambian cada segundo. Cuando alguno de los números sea 33 solo seguirán cambiando los otros dos. Utiliza Swing.

EJERCICIO 2

Desarrolla un programa que simule el servicio de impresión sabiendo que, cada usuario que imprime es representado por un hilo, y varios usuarios intentan imprimir de una impresora compartida. La impresora tiene un número limitado de impresiones a realizar (5).

Cuando un usuario imprime se debe asegurar que otro usuario no pueda imprimir al mismo tiempo. Para simular el trabajo de impresión puedes usar el siguiente código:

```
long startTime = System.nanoTime();
while (System.nanoTime() - startTime < 500_000_000)
{
    double x = Math.sqrt(Math.random());
}
```

Cuando se alcance el número máximo de impresiones, se mostrará un mensaje por consola indicándolo y finalizará el programa.

Solución

```
public class Impresora {
    private int impresiones;

    public Impresora(int impresiones) {
        this.impresiones = impresiones;
    }
}
```

```

    }

    // Método sincronizado para imprimir
    public synchronized boolean imprimir(String usuario) {
        if (impresiones > 0) {
            impresiones--;
            // Aquí va el código correspondiente al trabajo que realiza el hilo
            long startTime = System.nanoTime();
            while (System.nanoTime() - startTime < 500_000_000) {
                // Realizar cálculos para evitar que el compilador optimice el bucle
                double x = Math.sqrt(Math.random());
            }
            System.out.println(usuario + " ha realizado una impresión. Impresiones restantes: " + impresiones);

            return true;
        } else {
            System.out.println(usuario + ": No se puede seguir imprimiendo hasta que se cambie el toner.");
            return false;
        }
    }
}

// Método para verificar si aún quedan impresiones
public synchronized boolean impresoraDisponible() {
    return impresiones > 0;
}
}

public class ServidorDeImpresion {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        Impresora impresora = new Impresora(5);
        int contador = 1;

        while (impresora.impresoraDisponible()) {
            Usuario usuario = new Usuario(impresora, "Usuario " + contador++);

            usuario.start();
        }
        System.out.println("El toner está vacío.");
    }
}

```

```
    }  
}  
  
public class Usuario extends Thread{  
    private final Impresora impresora;  
    private final String nombre;  
  
    public Usuario(Impresora impresora, String nombre) {  
        this.impresora = impresora;  
        this.nombre = nombre;  
    }  
  
    @Override  
    public void run() {  
  
        if (!impresora.imprimir(nombre))  
            return;  
    }  
}
```
